Metodologie di Programmazione (A-L)

II semestre - a.a. 2023 – 2024

Parte 7 – Classi Interne**

a cura di Mattia Samory*



SAPIENZA
UNIVERSITÀ DI ROMA

*Tutti i diritti relativi al presente materiale didattico ed al suo contenuto sono riservati a Sapienza e ai suoi autori (o docenti che lo hanno prodotto). È consentito l'uso personale dello stesso da parte dello studente a fini di studio. Ne è vietata nel modo più assoluto la diffusione, duplicazione, cessione, trasmissione, distribuzione a terzi o al pubblico pena le sanzioni applicabili per legge.

**I crediti sulle slide di questo corso sono riportati nell'ultima slide

📣 Avvisi

- niente lezione questo venerdì, **19/4**
- condiviso un repository con il codice

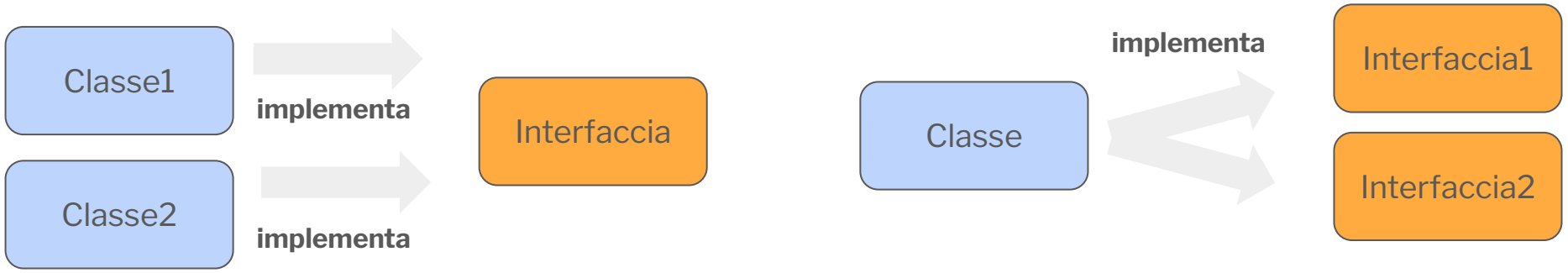
Letture

- Classi interne (Horstmann 10.5)
- [Opzionale] Mock (Horstmann 10.6)

🔄Recap

- interfacce in java
- Iterable ed Iterator

Oltre al sottoclasse, Java supporta un altro meccanismo di ereditarietà chiamato **sottotipizzazione**. La sottotipizzazione ci permette di specificare una relazione tra classi altrimenti non correlate, specificando un'**interfaccia**, **un insieme di metodi comuni** che ogni classe implementante deve contenere.



Method Summary

Methods

Modifier and Type	Method and Description
`Iterator<T>`	**`iterator`**`()` Returns an iterator over a set of elements of type T.

Method Summary

Methods

Modifier and Type	Method and Description
`boolean`	**`hasNext`**`()` Returns `true` if the iteration has more elements.
`E`	**`next`**`()` Returns the next element in the iteration.
`void`	**`remove`**`()` Removes from the underlying collection the last element returned by this iterator (optional operation).

 JukeBox

```java
import java.util.ArrayList;
import java.util.Iterator;

/**
 * Rappresenta un JukeBox di canzoni su cui si può iterare
 *
 * @author navigli
 */
public class Jukebox implements Iterable<Canzone>
{
    /**
     * Elenco delle canzoni
     */
    private ArrayList<Canzone> canzoni = new ArrayList<Canzone>();

    /**
     * Permette di aggiungere una canzone
     * @param c la canzone da aggiungere
     */
    public void addCanzone(Canzone c)
    {
        canzoni.add(c);
    }

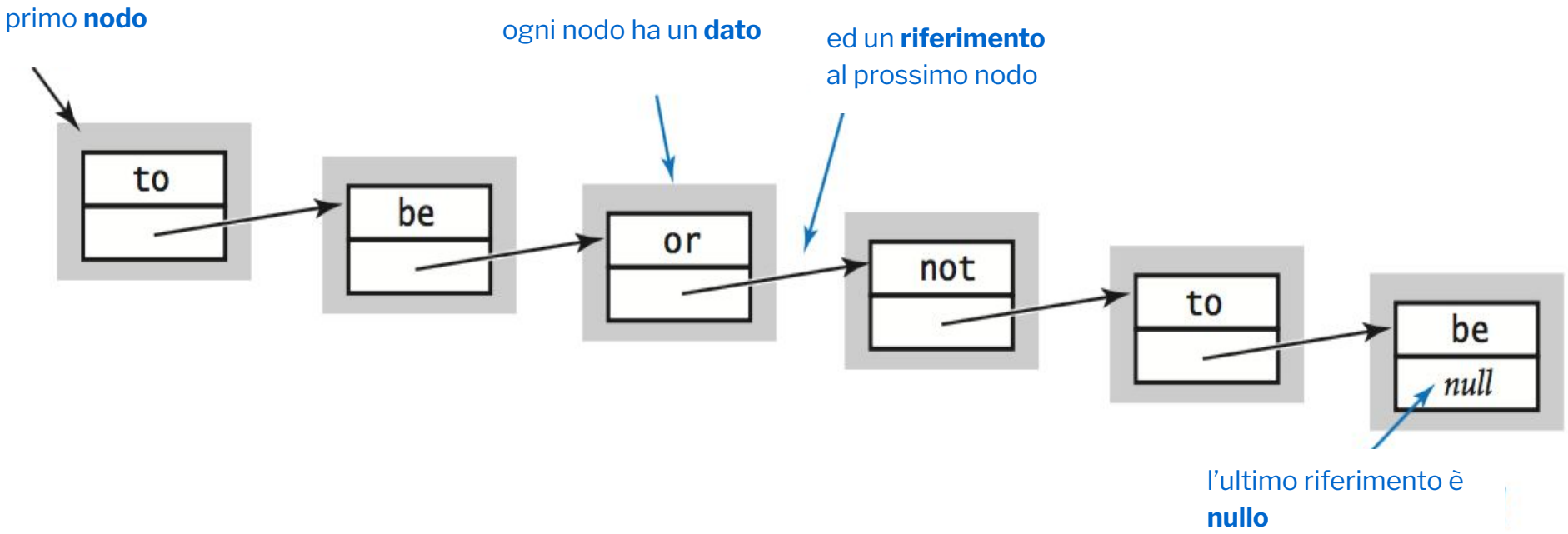
    @Override
    public Iterator<Canzone> iterator()
    {
        return canzoni.iterator();
    }
}
```



LinkedList

LinkedList

struttura dati ricorsiva: vuota, o un riferimento ad un **nodo.**



sketch operazioni

- inserimento in testa
- rimozione della testa
- attraversare la collezione
- inserimento in coda

👐TPS

Pensiamo a vantaggi e svantaggi di LinkedLists.

Per quali delle seguenti operazioni preferiremmo a LinkedList or an Array?

- aggiungere in mezzo alla collezione
- invertire l’ordine della collezione
- far crescere la collezione

🎯Obiettivi

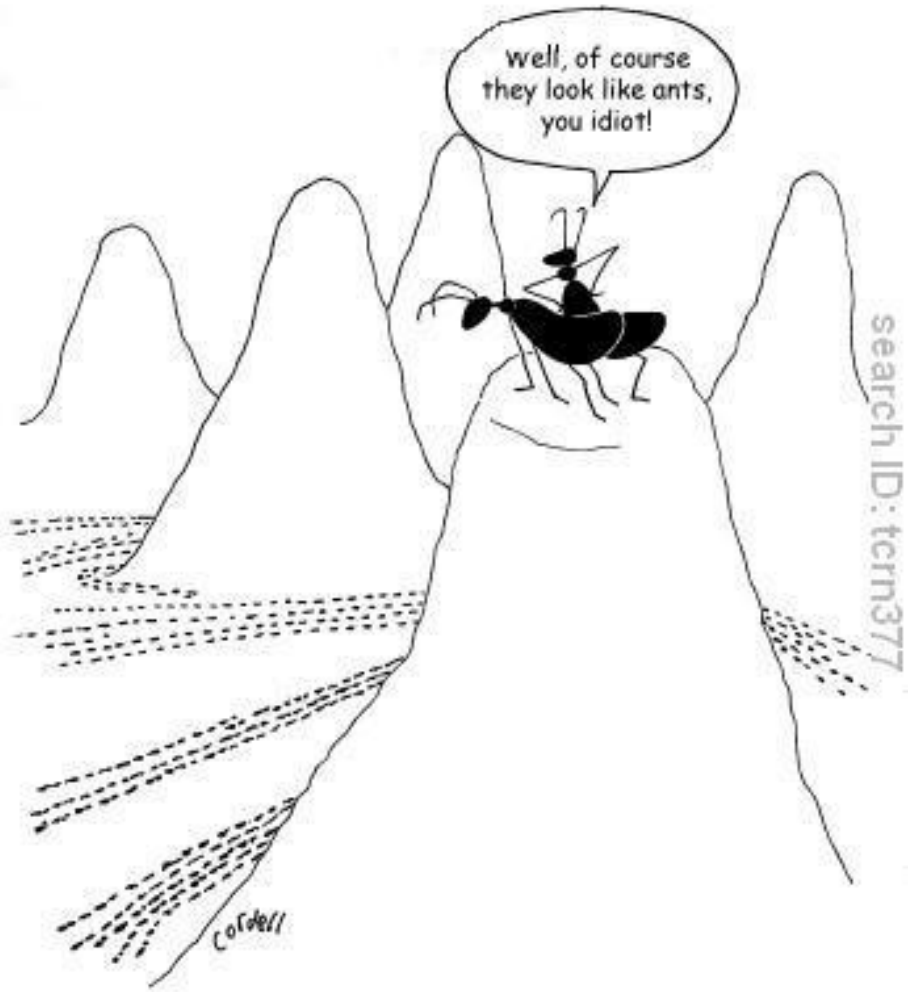
- classi interne
- classi anonime
- interfacce funzionali ed espressioni lambda

Classi annidate e interne

http://docs.oracle.com/javase/tutorial/java/javaOO/nested.html

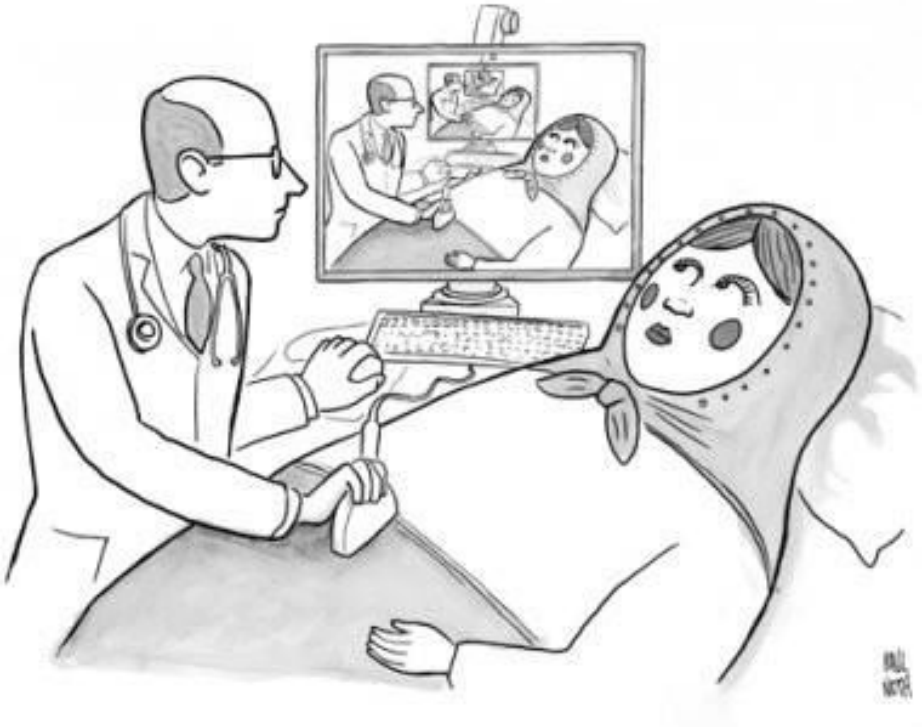
Classi top-level

- Le classi **usate finora** vengono dette top-level, cioè esse si trovano più in alto di tutte le altre e non sono contenute in altre classi
- Questo tipo di classi richiede un file .java dedicato con lo stesso nome della classe che esso contiene



Classi annidate (nested class)

- Java consente di scrivere classi all'interno di altre classi
- Le classi presenti all'interno sono chiamate classi annidate (nested class)
- Possono essere **static**
- Oppure **non-static**: in questo caso vengono dette **classi interne** (inner class)



Classi interne (inner class)

- Prima di poter creare un oggetto della classe interna è necessario istanziare la classe esterna (top-level) che la contiene
- Ciascuna classe interna, infatti, ha un riferimento implicito all'oggetto della classe che la contiene
- Dalla classe interna è possibile accedere a tutte le variabili e a tutti i metodi della classe esterna
- Come tutti i membri di una classe, le classi interne possono essere dichiarate public, protected o private

Classi interne (inner class)

- L'**accesso** a campi e metodi sia dell'oggetto della classe interna che dell'oggetto della classe esterna a cui è legato a doppio filo **avviene normalmente**
- Per disambiguare casi di ambiguità (es. campi con lo stesso nome nella classe interna ed esterna):
  - Se dalla classe interna viene usato soltanto **this** si fa riferimento ai campi e ai metodi di quella classe
  - Per far riferimento alla classe esterna è necessario far precedere **this** dal nome della classe esterna e il punto
- Per istanziare la classe interna dalla classe esterna è sufficiente utilizzare l'operatore new
- Per istanziare la classe interna a partire da un'altra classe si utilizza la sintassi riferimentoOggettoClasseEsterna.new ClasseInterna()

Classi interne (inner class)

```
public class Tastiera
{
    private String tipo;
    private Tasto[] tasti;

    public class Tasto
    {
        private char c;

        public Tasto(char c)
        {
            this.c = c;
        }

        public char premi()
        {
            return c;
        }

        public String toString()
        {
            return Tastiera.this.tipo + ": " + premi();
        }
    }

    public Tastiera(String tipo, char[] caratteri)
    {
        this.tipo = tipo;
        tasti = new Tasto[caratteri.length];

        for(int i=0; i<caratteri.length; i++)
            tasti[i] = new Tasto(caratteri[i]);
    }
}
```

La classe Tasto è una inner-class (quindi non statica)

Tasto ha accesso al campo (privato!) della classe esterna

Posso anche **accedere al campo direttamente tramite il nome** (tipo, invece di Tastiera.this.tipo)



Esercizio: Liste linkate di interi con classi interne

- Reimplementare le liste linkate di interi in modo da nascondere all'esterno l'implementazione del singolo elemento della lista
- Rendere iterabile la lista in modo che il seguente codice funzioni correttamente:

```
ListaLinkata l = new ListaLinkata(4, 8, 15,
16); l.add(23);
l.add(42);
// stampa gli elementi uno per ogni riga
for (int k : l) System.out.println(k);
// richiama l.toString()
System.out.println(l);
```

Classi annidate statiche (static nested class)

- Se invece la classe interna è statica allora essa non richiede l'esistenza di un oggetto appartenente alla classe esterna e non ha nemmeno un riferimento implicito ad essa
  - Come con i metodi statici, non può accedere allo stato dei singoli oggetti della classe esterna
- Da un punto di vista di comportamento, una classe annidata statica è equivalente ad una classe top-level inserita all'interno di un'altra classe top-level
- Sono accessibili tramite il nome della classe esterna che le contiene, secondo la forma
  new ClasseEsterna.ClasseAnnidataStatica()

Classi innestate statiche (static nested class)

```
public class Tastiera
{
    private String tipo;
    private Tasto[] tasti;

    public static class Tasto
    {
        private char c;

        public Tasto(char c)
        {
            this.c = c;
        }

        public char premi()
        {
            return c;
        }

        public String toString()
        {
            return Tastiera.this.tipo + ": " + premi();
        }
    }

    public Tastiera(String tipo, char[] caratteri)
    {
        this.tipo = tipo;
        tasti = new Tasto[caratteri.length];

        for(int i=0; i<caratteri.length; i++)
            tasti[i] = new Tasto(caratteri[i]);
    }
}
```

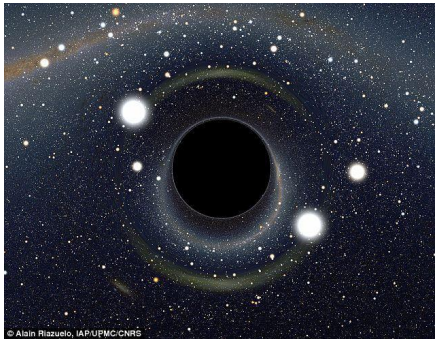
Errore in fase di compilazione: la classe static Tasto non ha accesso implicito ai membri della classe esterna

Ma la classe è visibile all'esterno

```
public class Computer
{
    public static void main(String[] args)
    {
        Tastiera.Tasto tasto = new Tastiera.Tasto('f');
        tasto.premi();
    }
}
```

Classi annidate: perché sono utili?

- Raggruppamento logico delle classi
  - Se una classe è **utile solo ad un'altra classe**, è logico inserirla al suo interno e tenere le due classi logicamente vicine
- Incrementa l'incapsulamento
  - Una classe B annidata in A può accedere ai membri di A (anche se privati), ma B può essere nascosta al mondo esterno
- Codice più leggibile e più facile da mantenere
  - La vicinanza spaziale è un fattore decisivo





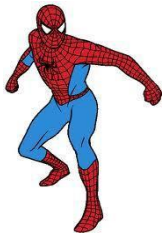
Esercizio: Disney vs. Marvel (1)

- Modellare uno scontro su campo tra personaggi Disney e personaggi Marvel
- Squadra Disney = Paperinik, Ciccionik, Superpippo
  - Paperinik, Ciccionik e Superpippo sono la **versione supereroe** dei corrispettivi personaggi (Paperino, Ciccio e Pippo)





- Squadra Marvel = Spiderman, La cosa, Magneto
  - Notare che Spiderman è la **versione supereroe** di Peter Parker, noto fotografo del Daily Bugle; La cosa e Magneto invece sono supereroi 24h







Esercizio: Disney vs. Marvel (2)

- I personaggi con una doppia vita devono esporre un'interfaccia DoppiaVita con i seguenti metodi:
  - assumiIdentitaSegreta(): consente di trasformarsi in supereroe
  - assumiIdentitaPubblica(): consente di assumere le sembianze 'umane'
- Ogni supereroe implementa l'interfaccia Supereroe con:
  - attacca(): sferra l'attacco specifico del supereroe
- Creare una partita su campo (ovvero una classe di test) in cui le due squadre si alternano in attacchi micidiali
  - Il nemico viene scelto casualmente dalla lista dei supereroi avversari



Esercizio: Disney vs. Marvel (3)

- Hint per versione **semplificata**: Senza utilizzare classi interne, ma solo interfacce ed ereditarietà
- Hint per versione **elaborata**: Per riflettere il fatto che nessuno è a conoscenza dei superpoteri in possesso dalle controparti umane, i supereroi, laddove possibile, dovrebbero estendere le classi umane corrispondenti ed essere implementati come loro classi interne private (ad es. Paperinik estende Paperino, ma solo Paperino potrà restituire un'istanza di Paperinik tramite l'interfaccia DoppiaVita)



🎯Obiettivi

- ✅ classi interne
- ❑ classi anonime
- ❑ interfacce funzionali ed espressioni lambda

Classi anonime

- E' possibile definire **classi anonime (ovvero senza nome)** che implementano un'interfaccia o estendono una classe
- Utilizzate esclusivamente per creare **un'unica istanza**
- Utili in determinati contesti (ad esempio, per creare un **iteratore al volo**)
- Sintassi:

```
TipoDaEstendere unicoRiferimentoAOggetto = new TipoDaEstendere()
{
  // codice della classe anonima (implementazione dell'interfaccia
  // o estensione della classe)
};
```

Esempio: l'interfaccia Formula

Consideriamo la seguente interfaccia:

```
public interface Formula

{
        double calculate(int a);
        default double sqrt(int a) { return Math.sqrt(a); }
}
```



Esempio:
una classe anonima che implementa l'interfaccia

```
Formula formula = new Formula()
{
        @Override
        public double calculate(int a)
        {
                return sqrt(a * 100);
        }
};

formula.calculate(100); // 100.0
formula.sqrt(16); // 4.0
```

🎯Obiettivi

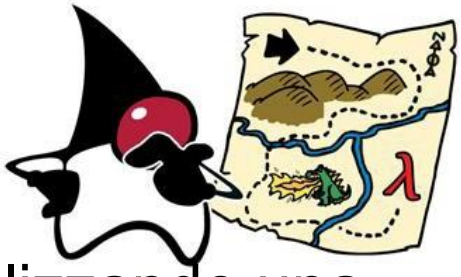
- ✅ classi interne
- ✅ classi anonime
- ❑ interfacce funzionali ed espressioni lambda

Interfacce funzionali

- In Java 8 è disponibile una nuova annotazione @FunctionalInterface
- L'annotazione garantisce che l'interfaccia sia dotata esattamente di un solo metodo astratto
- Ad esempio:

```
@FunctionalInterface
public interface Runnable
{
        void run();
}
```

Espressioni lambda



- In Java 8 è possibile specificare funzioni utilizzando una notazione molto compatta, le espressioni lambda:

```
() -> { System.out.println("hello, lambda!"); }
```

- Tali espressioni creano oggetti anonimi assegnabili a riferimenti a interfacce funzionali compatibili con l'intestazione (input/output) della funzione creata

```
Runnable r = () -> { System.out.println("hello, lambda!"); };
r.run(); // stampa "hello, lambda!"
```

Sintassi delle espressioni lambda

- Un'espressione lambda è definita come segue:

(tipo1 nome_param1, …, tipon nome_paramn) -> { codice della funzione }

- Il tipo dei parametri in input è opzionale, perché desunto dal contesto (a quale interfaccia funzionale facciamo riferimento?)
- Le parentesi tonde sono opzionali se in input abbiamo un solo parametro
- Le parentesi graffe intorno al codice della funzione sono opzionali se è costituito da una sola riga
  - Non è necessario return, se il codice è dato dall'espressione di ritorno



Espressioni lambda equivalenti



- (int a, int b) -> { return a+b; }
- (a, b) -> { return a+b; }
- (a, b) -> return a+b;
- (a, b) -> a+b;

- (String s) -> { return s.replace(‘ ‘, ‘_’); }
- (s) -> { return s.replace(‘ ‘, ‘_’); }
- s -> { return s.replace(‘ ‘, ‘_’); }
- s -> return s.replace(‘ ‘, ‘_’);
- s -> s.replace(‘ ‘, ‘_’);



Esempi di espressioni lambda

- (int a, int b) -> a+b
- a -> a*a
- () -> System.out.println("Hello, world!");
- s -> System.out.println(s);
- () -> 42;
- () -> 3.1428;
- (String s) -> s.length();
- s -> s.length();



Consideriamo nuovamente il nostro esempio

```
public interface Formula
{
        double calculate(int a);
}
```

Esempio di implementazione di Formula prima e dopo Java 8

In Java 7:

```
Formula formula = new Formula() {
        @Override
        public double calculate(int a)
        {
                return Math.sqrt(a * 100);
        }
    };
```

In Java 8:

```
Formula formula = a -> Math.sqrt(a*100);
Formula formula2 = a -> a*a;
Formula formula3 = a -> a-1;
```



Esempio: conversione da un tipo F a un tipo T

```
@FunctionalInterface
public interface Converter<F, T>
{
        T convert(F from);
}

Converter<String, Integer> converter = from -> Integer.valueOf(from);
Integer converted = converter.convert("123");
Converter<String, MyString> stringConverter = a -> new MyString(a);
MyString myString = stringConverter.convert(“123”);
```

Single Abstract Method (SAM) type

- Le interfacce funzionali sono di tipo SAM
- A ogni metodo che accetta un'interfaccia SAM, si può passare un'espressione lambda compatibile con l'unico metodo dell'interfaccia SAM
- Analogamente per un riferimento a un'interfaccia SAM



Differenze tra classi anonime ed espressioni lambda

- La parola chiave this:
  - **Classi anonime**: si riferisce all'oggetto anonimo
  - **Espressioni lambda**: si riferisce all'oggetto della classe che le racchiude
- La compilazione è differente:
  - **Classi anonime**: compilate come classi interne
  - **Espressioni lambda**: compilate come metodi privati invocati dinamicamente

Quando utilizzare le lambda?

- E' da consigliare l'impiego delle espressioni lambda **principalmente** quando il codice si scrive su una sola riga
- In alternativa, si preferisce un'implementazione mediante classe o classe anonima (o, vedremo più avanti, riferimenti a metodi)

🎯Obiettivi

- ✅ classi interne
- ✅ classi anonime
- ✅ interfacce funzionali ed espressioni lambda

Coding horror



- Due problemi qui:
  - If (bAcceso) ecc. si trasforma con l'operatore Elvis in una sola riga di codice
  - non usa costanti per "ACCESO" e "SPENTO"

```
public class ErroriComuni
{
    private boolean bAcceso;

    @Override
    public String toString()
    {
        if (bAcceso) return "ACCESO";
        else return "SPENTO";
    }
}
```

Coding horror



- Soluzione corretta:

```
public class ErroriComuni
{
    public static final String ACCESO = "acceso";
    public static final String SPENTO = "spento";

    private boolean bAcceso;

    @Override
    public String toString()
    {
        return bAcceso ? ACCESO : SPENTO;
    }
}
```

Coding horror



- Altre forme assolutamente ineleganti:

```
return bAcceso ? true : false;

if (bAcceso == true) return true;
else return false;
```

- Ancora:

```
if (bAcceso == true) System.out.println("vero");
if (bAcceso == false) System.out.println("falso");
```

- Si usa invece if (bAcceso) e, con l'operatore di negazione !, if (!bAcceso)

Coding horror

- Utilizzo del break o del return per uscire da un ciclo
  - Va evitato il booleano "d'appoggio" se non serve dopo!

```
public boolean cercaNO(int k, int... elenco)
{
    boolean trovato = false;

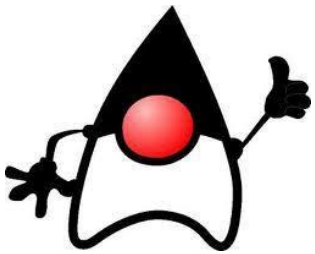
    for (int x : elenco)
        if (x == k) trovato = true;

    return trovato;
}
```




```
public boolean cercaOK(int k, int... elenco)
{
    for (int x : elenco)
        if (x == k) return true;

    return false;
}
```



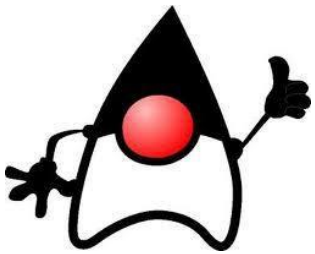
Coding horror

- Utilizzo del break o del return per uscire da un ciclo
  - Va evitato il booleano "d'appoggio" se non serve dopo!
- Esempio corretto d'uso di una variabile che memorizza il risultato dell'iterazione ma esce con il break appena il risultato richiesto è stato trovato:

```
public void cercaOK2(int k, int... elenco)
{
    boolean trovato = false;

    for (int x : elenco)
        if (x == k) { trovato = true; break; }

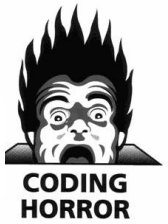
    // usa trovato
}
```



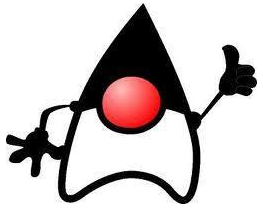
Coding horror

- Non posso istanziare interfacce:

```
List<Integer> l = new List<>();
```




```
List<Integer> l = new ArrayList<>();
```

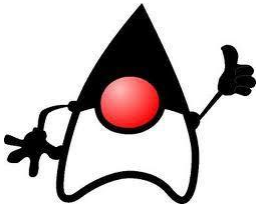


- Le collezioni (gli oggetti in generale...) vanno create prima di essere utilizzate:

```
ArrayList<Integer> l;
l.add(5);
```




```
ArrayList<Integer> l = new ArrayList<>();
l.add(5);
```



Coding horror

- Non posso utilizzare tipi primitivi tra parentesi angolari:

```
ArrayList<int> l;
```

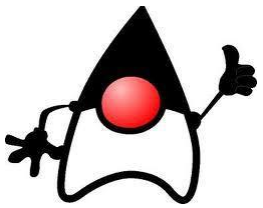


- Non posso assegnare un array a una lista:

```
List<Integer> l = new Integer[] { 1, 2, 3};
```




```
List<Integer> l = Arrays.asList(new Integer[] { 1, 2, 3});
```

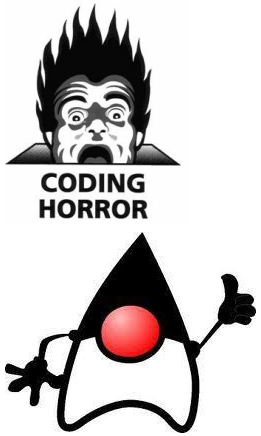


Coding horror

- Non posso accedere e modificare gli elementi della lista iterando con un foreach

```
// NON MODIFICA LA LISTA!
for (Integer k : l2) k += 5;

for (int k = 0; k < l.size(); k++)
    l.set(k, l.get(k)+5);
```

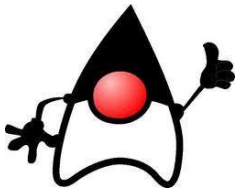


- Codice ridondante:

```
public int compareTo(Integer x)
{
    if (y > x) return 1;
    else if (y == x) return 0;
    else if (y < x) return -1;
}
```



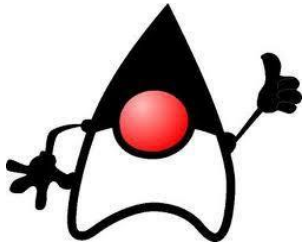

```
public int compareTo(Integer x)
{
    return y-x;
}
```



Coding horror

- Zucchero sintattico che non esiste in Java:
  - Non posso accedere agli elementi di una lista con le parentesi quadre: devo usare il metodo get
  - Non posso accedere ai caratteri di una stringa con le parantesi quadre: devo usare charAt
- Uso sempre l'operatore di post- (o pre-) incremento o assegnazione e somma:
- x += 1;
- x = x +1; x++;
- x = x + 5; x += 5





Credits

Le slide di questo corso sono basate su quelle utilizzate dal Prof. Faralli, le quali sono a loro volta una rielaborazione delle slide del Prof. Navigli, e frutto della revisione degli studenti borsisti della Facoltà di Ingegneria Informatica, Informatica e Statistica Mario Marra e Paolo Straniero.